

Name : Mahathi A J

RollNo: 241801148

SYSTEM CALLS PROGRAMMING

Aim: To experiment system calls using fork(), execlp() and pid() functions.

Algorithm:

1. Start

- Include the required header files (stdio.h and stdlib.h).

2. Variable Declaration

- Declare an integer variable pid to hold the process ID.

3. Create a Process

- Call the fork() function to create a new process. Store the return value in the pid variable:

If fork() returns:

-1: Forking failed (child process not created).

0: Process is the child process.

Positive integer: Process is the parent process.

4. Print Statement Executed Twice

- Print the statement:

Scss

Copy code

THIS LINE EXECUTED TWICE

(This line is executed by both parent and child processes after fork()).

5. Check for Process Creation Failure

- If pid == -1:

Print:

Copy code

CHILD PROCESS NOT CREATED

Exit the program using exit(0).

6. Child Process Execution

- If pid == 0 (child process):

Print:

Process ID of the child process using getpid().

Parent process ID of the child process using getppid().

7. Parent Process Execution

- If pid > 0 (parent process):

Print:

Process ID of the parent process using getpid().

Parent's parent process ID using getppid().

8. Final Print Statement

- Print the statement:

Name : Mahathi A J

RollNo: 241801148

Operating System

Objectivec

32Copy code

IT CAN BE EXECUTED TWICE

(This line is executed by both parent and child processes).

9. End

Program:

```
[root@localhost mahathi148]# vi syscall.c
[root@localhost mahathi148]# gcc syscall.c -o syscall
[root@localhost mahathi148]# ./syscall
THIS LINE EXECUTED TWICE
Parent Process
Parent PID: 35470
Parent's Parent PID: 35470
IT CAN BE EXECUTED TWICE
THIS LINE EXECUTED TWICE
Child process
Child PID: %D
Parent PID: 1927
IT CAN BE EXECUTED TWICE
[root@localhost mahathi148]#
```

```
mahathi@localhost:/home/mahathi/mahathi148 — /usr/bin/vim syscall.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(){
    int pid;
    pid = fork();
    printf("THIS LINE EXECUTED TWICE\n");
    if(pid == -1){
        printf("CHILD PROCESS NOT CLEARED\n");
        exit(0);
    }
    else if(pid == 0){
        printf("Child process\n");
        printf("Child PID: %D\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }
    else{
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getppid());
        printf("Parent's Parent PID: %d\n", getppid());
    }
    printf("IT CAN BE EXECUTED TWICE\n");
    return 0;
}
```